Retry

Retry · Publish event despite transient broker errors · Build with Waqar

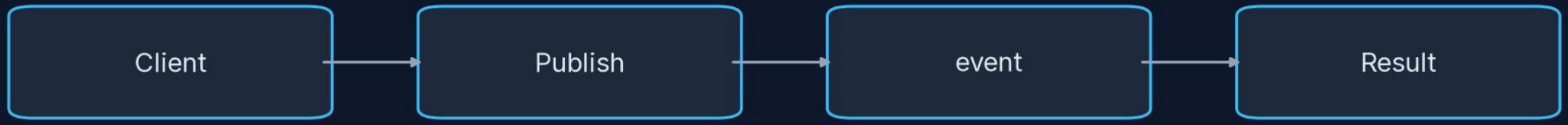
Today's Hook

- Problem: Publish event despite transient broker errors
- Pattern: Retry — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Publish event despite transient broker errors
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Retry: Publish event despite transient broker errors

Real-World Scenario

- Running example: Publish event despite transient broker errors
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.enterprise.retry;

import com.buildwithwaqar.patterns.enterprise.retry.Retry;
import java.util.concurrent.atomic.AtomicInteger;
/** Problem: publish to a flaky broker without failing the request on first blip. */
public final class UseCaseDemo {
  public static void main(String[] a) throws Exception {
    System.out.println("=== Retry problem: flaky broker publish ===");
    var n = new AtomicInteger();
    var retry = new Retry(3, e -> e instanceof IllegalStateException);
    System.out.println("result=" + retry.execute(() -> {
      if (n.incrementAndGet() < 3) throw new IllegalStateException("timeout");
      return "PUBLISHED";
    }));
    System.out.println("attempts=" + n.get());
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.enterprise.retry;

import java.util.concurrent.Callable;
import java.util.function.Predicate;

public final class Retry {
    private final int maxAttempts;
    private final Predicate<Exception> retryOn;

    public Retry(int maxAttempts, Predicate<Exception> retryOn) {
        this.maxAttempts = maxAttempts;
        this.retryOn = retryOn;
    }

    public <T> T execute(Callable<T> action) throws Exception {
        Exception last = null;
        for (int i = 1; i <= maxAttempts; i++) {
            try {
                return action.call();
            } catch (Exception e) {
                last = e;
                if (i == maxAttempts || !retryOn.test(e)) throw e;
            }
```

Solution (2)

```
            }
            throw last;
        }
    }
```

Runnable main

```java
package com.buildwithwaqar.patterns.enterprise.retry;

import com.buildwithwaqar.patterns.enterprise.retry.Retry;
import java.util.concurrent.atomic.AtomicInteger;
/** Problem: publish to a flaky broker without failing the request on first blip. */
public final class UseCaseDemo {
  public static void main(String[] a) throws Exception {
    System.out.println("=== Retry problem: flaky broker publish ===");
    var n = new AtomicInteger();
    var retry = new Retry(3, e -> e instanceof IllegalStateException);
    System.out.println("result=" + retry.execute(() -> {
      if (n.incrementAndGet() < 3) throw new IllegalStateException("timeout");
      return "PUBLISHED";
    }));
    System.out.println("attempts=" + n.get());
  }
}
```

Walkthrough Output

```
=== Retry problem: flaky broker publish ===
result=PUBLISHED
attempts=3
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Retry.

Related Patterns

- Contrast Retry with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Retry → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.